

CALL ME MAYBE

Build it. See it. Grow it.

The detailed learning blueprint

Six levels / Eighteen chapters / Clear handovers

BUILD / SEE / UNDERSTAND / VERIFY

Clear chapters. Numbered steps. Visible output.
Formal testing at the end of each chapter.

Prepared for Hayat / September 2026

How this blueprint is organized

Level > Chapter > Step > Substep. A level adds a visible capability to the same running program. Its chapters divide that capability into understandable features. Every chapter ends with testing. The final chapter of each level also checks that the level works together.

Implementation and observation happen before formal testing. Focused real runs and debugging are allowed while building. Tests establish which feature changed behavior; they are not the only evidence of progress.

This is a workflow and learning plan, not code or a predetermined implementation. No Need to Learn or What Are We Building Here sections are included, as requested.

Start from the work you already have

The uploaded snapshot already includes model access, vocabulary mapping, generation, prompting, JSON grammar, schema models, schema progress, masking, and tests. It lacks `src/__main__.py` while its Makefile expects module execution. Start by connecting and understanding existing work; do not rebuild completed pieces just to follow the new order.

Historical notes report outputs of -2 and 1 and earlier passing tests. Those observations do not verify the latest edits. Mark an item reused after inspecting its role; mark it verified only after relevant current checks.

The six-level route

1. A running journey, even when the answer is wrong.
2. JSON syntax guaranteed by decoding.
3. Runtime schema and complete function-call structure.
4. Correct meaning measured through real behavior.
5. Complete batch input/output and graceful errors.
6. Reproducible, documented, review-ready project.

Rules for every chapter

Before work, show the chapter goal, relevant files, program connection, decisions, and exact task. Keep the current position visible, for example Level 3 > Chapter 3.2 > Step 2 > Substep 2.1. Show the active checklist; do not flood every message with the full document.

Keep a decision record: question, chosen option, reason, affected files, required or provisional, and what would justify revisiting it. The subject governs requirements. The old blueprint and earlier AI decisions do not govern architecture.

If a test or real run reveals a defect, name the behavior, its cause, and the requirement before asking for a correction. Reopen the relevant numbered item, then return to the final testing step. Do not silently insert a new roadmap.

Level 1 - A running journey, even when the answer is wrong

Level structure

An entry point connects one request, runtime definitions, public model access, generation, and readable response output. Shared setup and per-request work have explicit boundaries. No JSON or semantic guarantee is claimed yet.

Visible output

A real request and its generated response are visible separately. The response may be prose, a scalar, an incorrect call, or malformed JSON. A generation failure is clearly identified rather than presented as success.

Goals and passing criteria

- Goal 1.1: One request reaches the real model through the entry point.
- Goal 1.2: Generated output is distinguishable from the original request and model instruction.
- Goal 1.3: Generation terminates or reports a clear failure; the learner can trace the files involved.
- Goal 1.4: Initialization and per-request responsibilities are recorded; later constraints have an identified connection point.

What this level does not yet promise

Correct JSON, function-call structure, or meaning.

Chapter 1.1 - Map the existing project and decide the boundaries

Place in the level: Chapter 1 of 3. Start from the previous level's working journey.

Feature outcome: Mark which existing constraints remain preserved while the baseline mode is observed.

Files to orient around: Entry point, llm_wrapper.py, vocab.py, generation.py, prompt.py, schema.py, and runtime input files. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Trace what already exists

- ☐ 1.1 Inspect the wrapper, vocabulary utility, generation loop, prompt builder, and current constraints.
- ☐ 1.2 Describe what each receives and returns using the actual snapshot; distinguish code from historical notes.
- ☐ 1.3 Mark reusable, incomplete, and disconnected pieces without demanding rewrites.

Step 2 - Agree on the first journey

- ☐ 2.1 Choose one real request and the runtime definitions to accompany it.
- ☐ 2.2 Map initialization, per-request processing, and displayed result; identify the missing entry point.
- ☐ 2.3 Record who owns original-request preservation, model context, generated tokens, and final records.

Step 3 - Make an actionable file map

- ☐ 3.1 Identify the proposed entry-point location and the existing files it will coordinate.
- ☐ 3.2 Agree on provisional interfaces and generation-stop behavior before editing callers.
- ☐ 3.3 Mark which existing constraints remain preserved while the baseline mode is observed.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Record what Chapter 1.2 receives, what it will change, and its exact first substep.

Chapter-specific verification focus: Existing-code/interface map agrees with the actual snapshot; recorded boundaries do not rely on hidden assumptions.

Chapter 1.2 - Connect the entry point and readable generation

Place in the level: Chapter 2 of 3. Carry forward Chapter 1.1; modify the same journey, not a parallel application.

Feature outcome: Display request, response, and any failure clearly in development output.

Files to orient around: Entry point, llm_wrapper.py, vocab.py, generation.py, prompt.py, schema.py, and runtime input files. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Connect shared setup

- ☐ 1.1 Use the supplied SDK through public access and existing wrapper.
- ☐ 1.2 Connect runtime definitions and vocabulary once where appropriate.
- ☐ 1.3 Keep setup failures visible and explain which file owns each boundary.

Step 2 - Connect one-request generation

- ☐ 2.1 Route original request and model context through the agreed boundary.
- ☐ 2.2 Reuse the existing loop; discuss how to observe a baseline without deleting completed constraint work.
- ☐ 2.3 Connect stopping and fresh request state without building all later-level guarantees.

Step 3 - Expose the response

- ☐ 3.1 Separate generated response from input context when decoding or displaying it.
- ☐ 3.2 Connect the minimal module entry point to the real generation path.
- ☐ 3.3 Display request, response, and any failure clearly in development output.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Record what Chapter 1.3 receives, what it will change, and its exact first substep.

Chapter-specific verification focus: Public model calls, generated-response boundary, fresh request handling, and explicit stop/failure outcomes.

Chapter 1.3 - Observe the baseline and stabilize the journey

Place in the level: Chapter 3 of 3. Carry forward Chapter 1.2; modify the same journey, not a parallel application.

Feature outcome: Carry forward the entry point and interfaces; record any still-provisional choice.

Files to orient around: Entry point, llm_wrapper.py, vocab.py, generation.py, prompt.py, schema.py, and runtime input files. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Run the first journey

- ☐ 1.1 Execute one real request through the connected entry point.
- ☐ 1.2 Capture the actual text and whether generation completed, stopped at a limit, or failed.
- ☐ 1.3 Trace the result back through the files rather than treating it as an unexplained model event.

Step 2 - Address only baseline blockers

- ☐ 2.1 Classify setup, interface, decoding, and stopping failures separately from wrong JSON or meaning.
- ☐ 2.2 Fix blocking baseline issues in explicit numbered tasks.
- ☐ 2.3 Defer JSON, schema, and semantic failures to their named levels.

Step 3 - Prepare the handover

- ☐ 3.1 Record a representative baseline output and its limitation.
- ☐ 3.2 Identify where JSON constraints will join the existing loop.
- ☐ 3.3 Carry forward the entry point and interfaces; record any still-provisional choice.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Check all of this level's passing criteria together through the real entry point. Record the evidence for each goal. Complete this level only when its chapters and level criteria are satisfied.

Chapter-specific verification focus: Actual baseline is reproducible; complete Level 1 journey works without claiming JSON or semantic correctness.

Level 1 handover

What now works: An entry point connects one request, runtime definitions, public model access, generation, and readable response output. Shared setup and per-request work have explicit boundaries. No JSON or semantic guarantee is claimed yet.

Evidence to keep: Representative real output, chapter verification results, goal-by-goal pass status, current decisions, and known limitations.

Why the next level exists: We have a complete path, but model output is not guaranteed to be JSON. Level 2 changes token eligibility inside this same path; it does not introduce a second application.

Next exact location: Level 2 > Chapter 2.1 > Step 1 > Substep 1.1.

Level 2 - JSON syntax guaranteed by decoding

Level structure

The running loop consults generic JSON grammar before selecting each token, advances state consistently, and recognizes complete output. Runtime function-call schema is still a later layer.

Visible output

A complete parseable JSON value. It could still be 1, a string, or an unrelated object: JSON syntax alone does not guarantee a function call.

Goals and passing criteria

- Goal 2.1: Completed real outputs parse as JSON.
- Goal 2.2: Illegal candidates are excluded before token selection, including multi-character candidates.
- Goal 2.3: State advancement, termination, and no-legal-candidate behavior are explicit.
- Goal 2.4: The same entry point works and earlier useful behavior remains available.

What this level does not yet promise

Correct function-call structure or meaning.

Chapter 2.1 - Prepare the generic grammar boundary

Place in the level: Chapter 1 of 3. Start from the previous level's working journey.

Feature outcome: Keep schema-specific behavior separate enough to observe the generic layer.

Files to orient around: `json_state.py`, `generic masking.py` behavior, `generation.py`, and the existing entry point. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Explain the baseline gap

- ☐ 1.1 Use an actual Level 1 output to identify the structural limitation.
- ☐ 1.2 Separate valid JSON syntax from function-call shape and correct meaning.
- ☐ 1.3 Locate existing JSON machine and generic masking responsibilities.

Step 2 - Decide candidate evaluation

- ☐ 2.1 Discuss validating every character of a candidate token against simulated state.
- ☐ 2.2 Discuss preserving live state when considering rejected candidates.
- ☐ 2.3 Discuss whitespace, escaped strings, numbers, nesting, and completion at the required complexity.

Step 3 - Prepare existing machinery

- ☐ 3.1 Inspect existing grammar transitions against the agreed role.
- ☐ 3.2 Identify concrete missing or conflicting behavior and implement only those changes.
- ☐ 3.3 Keep schema-specific behavior separate enough to observe the generic layer.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Record what Chapter 2.2 receives, what it will change, and its exact first substep.

Chapter-specific verification focus: Candidate simulation preserves live state; grammar handles required strings, escapes, numbers, and nesting.

Chapter 2.2 - Connect JSON constraints to token selection

Place in the level: Chapter 2 of 3. Carry forward Chapter 2.1; modify the same journey, not a parallel application.

Feature outcome: Keep diagnostics separate from the generated result.

Files to orient around: `json_state.py`, `generic_masking.py` behavior, `generation.py`, and the existing entry point. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Connect eligibility

- ☐ 1.1 Pass current grammar state and candidate text through generic masking.
- ☐ 1.2 Exclude illegal tokens before selection; preserve model ranking among surviving tokens.
- ☐ 1.3 Make the absence of legal candidates a clear outcome.

Step 2 - Connect state advancement

- ☐ 2.1 Advance live state using the selected token consistently with candidate evaluation.
- ☐ 2.2 Keep generated text and state synchronized across token boundaries.
- ☐ 2.3 Keep grammar state fresh for each request.

Step 3 - Connect completion

- ☐ 3.1 Stop on complete JSON according to the agreed generation boundary.
- ☐ 3.2 Distinguish a complete result from a limit-truncated result.
- ☐ 3.3 Keep diagnostics separate from the generated result.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Record what Chapter 2.3 receives, what it will change, and its exact first substep.

Chapter-specific verification focus: Illegal candidates are masked before selection; multi-character token advancement and completion agree.

Chapter 2.3 - Observe JSON behavior and prepare schema work

Place in the level: Chapter 3 of 3. Carry forward Chapter 2.2; modify the same journey, not a parallel application.

Feature outcome: Carry forward the running loop and grammar without duplicating them.

Files to orient around: `json_state.py`, `generic_masking.py` behavior, `generation.py`, and the existing entry point. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Inspect real constrained output

- ☐ 1.1 Run the same journey with generic JSON constraints.
- ☐ 1.2 View the complete text and parse it for inspection.
- ☐ 1.3 Explain why parseability alone does not establish project correctness.

Step 2 - Investigate structural blockers

- ☐ 2.1 Trace a malformed or incomplete result to candidate checks, advancement, or stopping.
- ☐ 2.2 Fix demonstrated JSON-layer defects rather than tuning semantic behavior.
- ☐ 2.3 Keep a clear record of failures belonging to the schema level.

Step 3 - Prepare the handover

- ☐ 3.1 Record what is now guaranteed and what is not.
- ☐ 3.2 Identify schema-state and runtime-definition integration points.
- ☐ 3.3 Carry forward the running loop and grammar without duplicating them.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Check all of this level's passing criteria together through the real entry point. Record the evidence for each goal. Complete this level only when its chapters and level criteria are satisfied.

Chapter-specific verification focus: Repeated completed real outputs parse; truncation is reported clearly; all Level 2 criteria hold.

Level 2 handover

What now works: The running loop consults generic JSON grammar before selecting each token, advances state consistently, and recognizes complete output. Runtime function-call schema is still a later layer.

Evidence to keep: Representative real output, chapter verification results, goal-by-goal pass status, current decisions, and known limitations.

Why the next level exists: We can trust JSON syntax, but not the required record shape. Level 3 adds constraints from the output contract and runtime function definitions while keeping generic grammar responsible for syntax.

Next exact location: Level 3 > Chapter 3.1 > Step 1 > Substep 1.1.

Level 3 - Runtime schema and complete function-call structure

Level structure

Generic JSON constraints compose with runtime schema constraints. Valid function names, required parameter names and types, outer-record ownership, and exact original-request preservation have agreed responsibilities.

Visible output

A record with original prompt, available function name, and parameters matching that function's runtime schema. It may still select the wrong available function or wrong argument values.

Goals and passing criteria

- Goal 3.1: Completed records contain exactly prompt, name, and parameters, with required types.
- Goal 3.2: Original request content is preserved, including quotes and special characters.
- Goal 3.3: Function names and parameter requirements come from runtime definitions; no unknown or duplicate keys.
- Goal 3.4: Required arguments and supported runtime types are enforced before selection, while semantic choices remain model-led.

What this level does not yet promise

Correct meaning: a permitted function or value may be wrong.

Chapter 3.1 - Decide record ownership and runtime contracts

Place in the level: Chapter 1 of 3. Start from the previous level's working journey.

Feature outcome: Record affected schema, masking, generation, prompt, and entry-point boundaries.

Files to orient around: `schema.py`, `schema_state.py`, `schema masking.py` behavior, `generation.py`, `prompt.py`, and record assembly boundary. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Read the complete contract

- ☐ 1.1 Inspect the subject's output requirements and actual definition models.
- ☐ 1.2 Discuss whether generation produces the entire record or a defined portion assembled into the record; justify against the subject.
- ☐ 1.3 Record key ordering, original-request ownership, and escaping responsibilities before tiny edits.

Step 2 - Connect runtime definitions

- ☐ 2.1 Derive function candidates from the loaded definitions.
- ☐ 2.2 Explain how a chosen function supplies its parameter schema.
- ☐ 2.3 Identify supported types and requiredness from the contract, not only five example functions.

Step 3 - Map state responsibilities

- ☐ 3.1 Distinguish generic grammar state from schema progress.
- ☐ 3.2 Agree on how outer keys, selected function, partial keys, and completed parameters are represented.
- ☐ 3.3 Record affected schema, masking, generation, prompt, and entry-point boundaries.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Record what Chapter 3.2 receives, what it will change, and its exact first substep.

Chapter-specific verification focus: Runtime models and record ownership match the subject; original-request serialization is explicit.

Chapter 3.2 - Constrain record structure and function candidates

Place in the level: Chapter 2 of 3. Carry forward Chapter 3.1; modify the same journey, not a parallel application.

Feature outcome: Keep schema and grammar advancement synchronized.

Files to orient around: `schema.py`, `schema_state.py`, `schema masking.py` behavior, `generation.py`, `prompt.py`, and record assembly boundary. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Connect outer structure

- ☐ 1.1 Apply the agreed record-generation boundary and required outer fields.
- ☐ 1.2 Preserve the original request according to the recorded ownership and serialization decision.
- ☐ 1.3 Handle field progression without silently assuming an unagreed ordering.

Step 2 - Connect function candidates

- ☐ 2.1 Restrict candidate prefixes to runtime names across token boundaries.
- ☐ 2.2 Discuss exact matches, longer names sharing a prefix, and completion.
- ☐ 2.3 Ensure logits decide among legal names rather than request keywords.

Step 3 - Connect selected-function handover

- ☐ 3.1 Record the completed function choice at the right boundary.
- ☐ 3.2 Make that choice's runtime parameter definition available to the next stage.
- ☐ 3.3 Keep schema and grammar advancement synchronized.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Record what Chapter 3.3 receives, what it will change, and its exact first substep.

Chapter-specific verification focus: Outer keys, escaping, prefix-sharing names, function membership, and schema/grammar handover.

Chapter 3.3 - Constrain parameters and complete the record

Place in the level: Chapter 3 of 3. Carry forward Chapter 3.2; modify the same journey, not a parallel application.

Feature outcome: Record the structurally valid but semantically imperfect baseline for Level 4.

Files to orient around: `schema.py`, `schema_state.py`, `schema masking.py` behavior, `generation.py`, `prompt.py`, and record assembly boundary. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Connect parameter keys

- ☐ 1.1 Derive remaining legal keys from the selected function.
- ☐ 1.2 Handle split keys and prevent unknown or repeated parameters.
- ☐ 1.3 Know when all required parameters are present, including applicable zero-parameter cases.

Step 2 - Connect types and completion

- ☐ 2.1 Restrict values according to the supported runtime types.
- ☐ 2.2 Leave actual value extraction to the model, not request-specific Python rules.
- ☐ 2.3 Permit closing only when the agreed record requirements are satisfied.

Step 3 - Observe full structural output

- ☐ 3.1 Run a real request and inspect the complete record through the existing entry point.
- ☐ 3.2 Investigate shape, original-request, type, and state failures separately from semantic mistakes.
- ☐ 3.3 Record the structurally valid but semantically imperfect baseline for Level 4.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Check all of this level's passing criteria together through the real entry point. Record the evidence for each goal. Complete this level only when its chapters and level criteria are satisfied.

Chapter-specific verification focus: Required/unknown/duplicate parameters, supported types, completion, changed definitions, and full real records.

Level 3 handover

What now works: Generic JSON constraints compose with runtime schema constraints. Valid function names, required parameter names and types, outer-record ownership, and exact original-request preservation have agreed responsibilities.

Evidence to keep: Representative real output, chapter verification results, goal-by-goal pass status, current decisions, and known limitations.

Why the next level exists: Structure is dependable, but a legal choice can have the wrong meaning. Level 4 investigates context and model decisions without replacing them with Python heuristics.

Next exact location: Level 4 > Chapter 4.1 > Step 1 > Substep 1.1.

Level 4 - Correct meaning measured through real behavior

Level structure

Dynamic model instructions provide usable function context. Structural constraints remain intact; function choice and argument extraction remain model decisions. Changes are evaluated against explicit expected results.

Visible output

Structurally valid calls that select the intended functions and preserve the requested argument values. Failures are categorized so accuracy is measured rather than inferred from one good example.

Goals and passing criteria

- Goal 4.1: Achieve the subject's 90%+ correct function selection and argument extraction target on the evaluation set.
- Goal 4.2: Retain 100% JSON/schema validity for successful generated records.
- Goal 4.3: Work with changed runtime functions and unfamiliar names rather than sample-specific mappings.
- Goal 4.4: Record observed accuracy and context decisions; do not claim universal semantic certainty.

What this level does not yet promise

Complete batch/file/error handling and final packaging.

Chapter 4.1 - Diagnose the semantic baseline

Place in the level: Chapter 1 of 3. Start from the previous level's working journey.

Feature outcome: Record the choice as an experiment, not an architectural law.

Files to orient around: prompt.py, generation.py, runtime definitions, and schema/masking only when a demonstrated integration issue requires it. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Inspect a wrong legal call

- ☐ 1.1 Compare actual function and values with the request's intended meaning.
- ☐ 1.2 Confirm the correct function and values were not accidentally excluded by structural constraints.
- ☐ 1.3 Classify function-choice, value-extraction, context, and integration failures.

Step 2 - Inspect model context

- ☐ 2.1 Read the current prompt builder and the actual text reaching the model.
- ☐ 2.2 Discuss descriptions, parameter information, instruction boundaries, and original-request serialization.
- ☐ 2.3 Revisit the friend's screenshots if available as inspiration; do not copy or pretend unavailable material was read.

Step 3 - Choose focused experiments

- ☐ 3.1 State which observed failure a proposed change addresses.
- ☐ 3.2 Choose a small context change with a predicted visible effect.
- ☐ 3.3 Record the choice as an experiment, not an architectural law.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Record what Chapter 4.2 receives, what it will change, and its exact first substep.

Chapter-specific verification focus: Observed wrong calls are classified accurately; model sees the actual runtime context.

Chapter 4.2 - Improve context while preserving model-led decisions

Place in the level: Chapter 2 of 3. Carry forward Chapter 4.1; modify the same journey, not a parallel application.

Feature outcome: Keep structural guarantees and original-request handling visible.

Files to orient around: `prompt.py`, `generation.py`, runtime definitions, and schema/masking only when a demonstrated integration issue requires it. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Implement the selected change

- ☐ 1.1 Identify the relevant prompt or integration file before work.
- ☐ 1.2 Let Hayat implement dynamic context and appropriate JSON escaping.
- ☐ 1.3 Avoid hardcoded names, sample-specific extraction, or extra dependencies.

Step 2 - Observe representative requests

- ☐ 2.1 Run a small set of actual requests and view complete records.
- ☐ 2.2 Compare the result with the prediction and record what changed.
- ☐ 2.3 Investigate demonstrated failures without silently redesigning earlier layers.

Step 3 - Keep useful improvements

- ☐ 3.1 Retain changes supported by behavior and remove unjustified experiments deliberately.
- ☐ 3.2 Record which failure categories remain.
- ☐ 3.3 Keep structural guarantees and original-request handling visible.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Record what Chapter 4.3 receives, what it will change, and its exact first substep.

Chapter-specific verification focus: Prompt changes address predicted failures and do not break structural validity or dynamic definitions.

Chapter 4.3 - Establish measurable semantic performance

Place in the level: Chapter 3 of 3. Carry forward Chapter 4.2; modify the same journey, not a parallel application.

Feature outcome: Prepare to reopen a specific implementation item if evaluation exposes a failure.

Files to orient around: `prompt.py`, `generation.py`, runtime definitions, and schema/masking only when a demonstrated integration issue requires it. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Define expected results

- ☐ 1.1 List expected function and argument values for supplied prompts.
- ☐ 1.2 Define success as both the correct function and correct required values.
- ☐ 1.3 Separate semantic scores from syntax and schema validity.

Step 2 - Prepare generalization cases

- ☐ 2.1 Use changed/reordered definitions and unfamiliar names at comparable complexity.
- ☐ 2.2 Include meaningful strings, numbers, quoting, ambiguity, and multi-parameter requests.
- ☐ 2.3 Avoid inventing unsupported contract requirements.

Step 3 - Record the semantic handover

- ☐ 3.1 Observe final representative outputs before formal evaluation.
- ☐ 3.2 Record remaining uncertainty and the generation boundary Level 5 will reuse.
- ☐ 3.3 Prepare to reopen a specific implementation item if evaluation exposes a failure.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Check all of this level's passing criteria together through the real entry point. Record the evidence for each goal. Complete this level only when its chapters and level criteria are satisfied.

Chapter-specific verification focus: All supplied prompts plus changed-function cases: correct function AND arguments, 90%+ semantic accuracy, and intact JSON/schema guarantees.

Level 4 handover

What now works: Dynamic model instructions provide usable function context. Structural constraints remain intact; function choice and argument extraction remain model decisions. Changes are evaluated against explicit expected results.

Evidence to keep: Representative real output, chapter verification results, goal-by-goal pass status, current decisions, and known limitations.

Why the next level exists: One-request behavior is now understood and measured. Level 5 expands the same boundary to input files, multiple requests, and durable output; it does not rebuild the generation engine.

Next exact location: Level 5 > Chapter 5.1 > Step 1 > Substep 1.1.

Level 5 - Complete batch input/output and graceful errors

Level structure

The module entry point accepts required optional path arguments, loads inputs and shared resources, runs each request with fresh state, and writes the required JSON array. Failure policies are explicit.

Visible output

The required result file contains one successful record per processed request under the agreed subject-compatible failure policy. Missing or invalid inputs and generation/output failures produce clear diagnostics instead of unexpected crashes.

Goals and passing criteria

- Goal 5.1: Required module invocation and custom path arguments work.
- Goal 5.2: Defaults follow the subject's mandatory output format and supplied input locations.
- Goal 5.3: Multiple requests do not leak grammar or schema state into one another.
- Goal 5.4: Malformed/missing files and other expected failures are handled clearly.
- Goal 5.5: Accuracy and validity remain intact; supplied test batch meets the subject's under-five-minute target on standard hardware.

What this level does not yet promise

Final packaging, tooling, and documentation compliance.

Chapter 5.1 - Define external contracts and failure policy

Place in the level: Chapter 1 of 3. Start from the previous level's working journey.

Feature outcome: Agree on output writing and what happens when a request or write fails.

Files to orient around: Entry point, input validation boundary, one-request generation boundary, output writer, and definition models. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Confirm paths and invocation

- ☐ 1.1 Read the subject's module command and three optional path arguments.
- ☐ 1.2 Confirm default inputs and data/output/function_calling_results.json; distinguish the custom filename in the usage example.
- ☐ 1.3 Connect argument handling to the existing entry point.

Step 2 - Decide input validation

- ☐ 2.1 Agree on parsing and validation boundaries for prompts and definitions.
- ☐ 2.2 Discuss missing files, malformed JSON, wrong input shapes, and unsupported definitions.
- ☐ 2.3 Choose clear diagnostics without inventing result records that violate output schema.

Step 3 - Decide batch and output ownership

- ☐ 3.1 Initialize expensive shared resources once.
- ☐ 3.2 Create fresh request-specific state for each request.
- ☐ 3.3 Agree on output writing and what happens when a request or write fails.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Record what Chapter 5.2 receives, what it will change, and its exact first substep.

Chapter-specific verification focus: Default/custom paths, missing and malformed inputs, and subject-compatible failure policy.

Chapter 5.2 - Expand the journey to multiple requests and file output

Place in the level: Chapter 2 of 3. Carry forward Chapter 5.1; modify the same journey, not a parallel application.

Feature outcome: Manage files and necessary directory creation according to the agreed behavior.

Files to orient around: Entry point, input validation boundary, one-request generation boundary, output writer, and definition models. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Connect loading

- ☐ 1.1 Use the agreed paths and validated inputs.
- ☐ 1.2 Reuse current definition models and resource initialization.
- ☐ 1.3 Keep loading separate from request-specific generation.

Step 2 - Connect batch processing

- ☐ 2.1 Call the existing one-request boundary for every input request.
- ☐ 2.2 Reset grammar and schema state for each call.
- ☐ 2.3 Collect complete results without mixing development diagnostics into result data.

Step 3 - Connect output writing

- ☐ 3.1 Write the required JSON array to the chosen path.
- ☐ 3.2 Preserve exact request content and required record keys.
- ☐ 3.3 Manage files and necessary directory creation according to the agreed behavior.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Record what Chapter 5.3 receives, what it will change, and its exact first substep.

Chapter-specific verification focus: Multiple requests, fresh state, collected records, output path and valid JSON array.

Chapter 5.3 - Complete errors and observe the actual application

Place in the level: Chapter 3 of 3. Carry forward Chapter 5.2; modify the same journey, not a parallel application.

Feature outcome: Record the complete journey for final packaging and documentation.

Files to orient around: Entry point, input validation boundary, one-request generation boundary, output writer, and definition models. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Handle boundary failures

- ☐ 1.1 Implement agreed handling for loading, model setup, generation, and output writing.
- ☐ 1.2 Make messages actionable without hiding failures as successful calls.
- ☐ 1.3 Manage resources consistently and avoid broad exception swallowing.

Step 2 - Run real default and custom paths

- ☐ 2.1 Run the documented module invocation with supplied inputs.
- ☐ 2.2 Inspect the generated file and request-to-result correspondence.
- ☐ 2.3 Run custom paths and inspect the actual location used.

Step 3 - Observe representative failures

- ☐ 3.1 Try representative missing/malformed input and output failures.
- ☐ 3.2 Explain which boundary handled each failure.
- ☐ 3.3 Record the complete journey for final packaging and documentation.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Check all of this level's passing criteria together through the real entry point. Record the evidence for each goal. Complete this level only when its chapters and level criteria are satisfied.

Chapter-specific verification focus: Graceful boundary failures, actual invocation, all Level 5 criteria, and batch performance under the subject target.

Level 5 handover

What now works: The module entry point accepts required optional path arguments, loads inputs and shared resources, runs each request with fresh state, and writes the required JSON array. Failure policies are explicit.

Evidence to keep: Representative real output, chapter verification results, goal-by-goal pass status, current decisions, and known limitations.

Why the next level exists: The full application runs. Level 6 completes tooling, documentation, packaging, and final evidence so this behavior is reproducible and explainable during review.

Next exact location: Level 6 > Chapter 6.1 > Step 1 > Substep 1.1.

Level 6 - Reproducible, documented, review-ready project

Level structure

The complete application has compliant dependencies, public SDK use, required validation/tooling, documentation, repository contents, and recorded verification evidence. Hayat can explain the actual implementation.

Visible output

The same function-call output and clear failure behavior, now reproducible through documented setup and invocation. README and verification records reflect measured behavior rather than aspirational claims.

Goals and passing criteria

- Goal 6.1: Required install/run/debug/clean/lint behavior and uv sync work.
- Goal 6.2: Required flake8 and mypy checks pass; typing, docstrings, and Pydantic requirements are addressed.
- Goal 6.3: README and repository contents meet the subject, including AI-use disclosure and generated-output exclusion.
- Goal 6.4: Final checks substantiate validity, dynamic schema handling, semantic accuracy, speed, and graceful errors.
- Goal 6.5: Hayat can trace a request and explain important design choices and limitations.

What this level does not yet promise

No unchecked requirement is treated as complete.

Chapter 6.1 - Audit compliance and resolve concrete gaps

Place in the level: Chapter 1 of 3. Start from the previous level's working journey.

Feature outcome: Prepare exact setup and invocation instructions.

Files to orient around: Actual implementation files, pyproject.toml, uv.lock, Makefile, README.md, and repository ignore rules. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Audit implementation requirements

- ☐ 1.1 Review Python version, allowed dependencies, Qwen3-0.6B compatibility, and public SDK access.
- ☐ 1.2 Review the subject's all-classes Pydantic requirement and its implications for existing state classes.
- ☐ 1.3 Identify concrete typing, docstring, resource, and error-handling gaps.

Step 2 - Plan scoped corrections

- ☐ 2.1 Distinguish explicit requirements from optional style choices.
- ☐ 2.2 Identify affected files and the behavior that must remain stable.
- ☐ 2.3 Apply necessary corrections in visible numbered tasks without a blanket redesign.

Step 3 - Record readiness evidence

- ☐ 3.1 List requirements already supported by observed behavior.
- ☐ 3.2 Record remaining checks and uncertainties honestly.
- ☐ 3.3 Prepare exact setup and invocation instructions.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Record what Chapter 6.2 receives, what it will change, and its exact first substep.

Chapter-specific verification focus: Required dependencies/public SDK/Pydantic/typing changes preserve existing behavior.

Chapter 6.2 - Complete packaging and documentation

Place in the level: Chapter 2 of 3. Carry forward Chapter 6.1; modify the same journey, not a parallel application.

Feature outcome: Avoid claiming measurements that have not been taken.

Files to orient around: Actual implementation files, pyproject.toml, uv.lock, Makefile, README.md, and repository ignore rules. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Complete the execution package

- ☐ 1.1 Confirm src, dependency files, supplied SDK, input examples, and required Makefile targets.
- ☐ 1.2 Confirm required lint flags and module execution path.
- ☐ 1.3 Exclude generated output and irrelevant artifacts as the subject requires.

Step 2 - Write the English README

- ☐ 2.1 Use the required italicized curriculum/login first line.
- ☐ 2.2 Include Description, Instructions, and Resources with AI-use disclosure.
- ☐ 2.3 Explain algorithm, design choices, measured performance, challenges, testing, and examples.

Step 3 - Check documentation against behavior

- ☐ 3.1 Follow the documented run and inspect its output.
- ☐ 3.2 Ensure commands, paths, guarantees, and limitations match reality.
- ☐ 3.3 Avoid claiming measurements that have not been taken.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Record what Chapter 6.3 receives, what it will change, and its exact first substep.

Chapter-specific verification focus: Documented setup, required repository contents, Makefile behavior, and accurate README claims.

Chapter 6.3 - Demonstrate understanding and verify the full system

Place in the level: Chapter 3 of 3. Carry forward Chapter 6.2; modify the same journey, not a parallel application.

Feature outcome: Finish with the full testing step below; leave unresolved gaps visible.

Files to orient around: Actual implementation files, pyproject.toml, uv.lock, Makefile, README.md, and repository ignore rules. Inspect the actual interfaces before deciding which of these needs an edit.

Step 1 - Explain the journey

- ☐ 1.1 Trace one actual request across actual files in Hayat's own words.
- ☐ 1.2 Explain grammar constraints, runtime schema constraints, and model-led decisions.
- ☐ 1.3 Explain important provisional choices and why they became final choices.

Step 2 - Demonstrate behavior

- ☐ 2.1 Run one successful example through the documented entry point.
- ☐ 2.2 Demonstrate one gracefully handled failure.
- ☐ 2.3 Discuss how a small requested change would affect the current architecture.

Step 3 - Prepare final verification

- ☐ 3.1 Check repository and setup instructions against the subject.
- ☐ 3.2 Gather previous measured results and identify any checks invalidated by new edits.
- ☐ 3.3 Finish with the full testing step below; leave unresolved gaps visible.

Step 4 - Testing, last

- ☐ 4.1 After implementation, run the chapter's new behavior and capture its visible result. Separate expected early-level limitations from defects in this chapter.
- ☐ 4.2 Check the chapter's feature boundary using relevant existing tests; add focused missing checks only where they verify meaningful behavior.
- ☐ 4.3 Run checks for earlier behavior touched by this feature. A failure must identify the changed boundary; do not replace diagnosis with an ever-growing suite.
- ☐ 4.4 Record implemented, observed running, and verified separately. Reopen the relevant substep if a check fails, then repeat affected checks.
- ☐ 4.5 Check all of this level's passing criteria together through the real entry point. Record the evidence for each goal. Complete this level only when its chapters and level criteria are satisfied.

Chapter-specific verification focus: Full regression and subject verification, mandatory flake8/mypy flags, uv sync, accuracy, validity, performance, errors, documentation, and packaging.

Level 6 handover

What now works: The complete application has compliant dependencies, public SDK use, required validation/tooling, documentation, repository contents, and recorded verification evidence. Hayat can explain the actual implementation.

Evidence to keep: Representative real output, chapter verification results, goal-by-goal pass status, current decisions, and known limitations.

Why the next level exists: The level closes the project only when evidence meets the criteria. An unresolved requirement stays visible and reopens its specific task; it is not erased by marking the final checklist complete.

Next exact location: Project completion record, or the specifically reopened requirement if a final check fails.

Updated companion instructions - use with this blueprint

This section supersedes the structure and pacing rules in the earlier companion prompt. The earlier one-level-equals-one-chapter rule no longer applies.

Help Hayat understand and implement her own code. Do not write complete project implementations or edit the repository without explicit authorization. Answer direct conceptual and syntax questions clearly; do not require a hint ladder or refuse teaching because she already read something.

Always use Level > Chapter > Step > Substep. Restore the exact position at session start. Introduce the active chapter's goal and checklist, then orient the current step: purpose, files, program connection, decisions, task, and expected observable result. Let her attempt it and wait. A meaningful task may include multiple connected lines; avoid line-by-line permission gates.

Keep formal testing as the last step of every chapter. Run and inspect output during development. Reuse relevant tests afterward and check earlier behavior affected by the new feature. The last chapter checks the complete level against its stated criteria. Do not advance a level on a historical test count.

Explain failures through observed behavior, cause, and requirement. Separate blockers from later-level concerns and optional polish. Surface architectural choices before local implementation. Update checklists explicitly if new evidence changes the plan; the user can change direction.

Preserve model-led function selection, dynamic definitions, before-selection constraints, original-request content, and the subject's final requirements. Do not substitute keyword heuristics, post-hoc retries, or sample-specific rules. Treat architecture choices as choices unless the subject explicitly requires them.

At each handoff record current location, completed/reused items, actual observed output, verification status, decisions, known failures, and the exact next substep with relevant files. No repeated research prerequisite, no mandatory restart, and no pretending unavailable reference screenshots were read.

Future blueprint notes - keep for later projects

These are Hayat's future preferences, not extra tasks for the current Call Me Maybe plan.

- Include an overview section that explains the whole project journey and how the parts fit together. The overview was especially helpful during Call Me Maybe.
- Include a Need to Learn section, linked to the relevant levels and chapters.
- Include a What Are We Building Here section with clear purpose and expected behavior.
- Add a daily calendar for each level, provisionally based on approximately six-hour working days. Include breaks and realistic scope rather than assuming six uninterrupted hours of implementation.
- When planning alongside university, use the actual university schedule and workload to adjust available time. Do not invent dates or assume six hours are available every day.
- Consider applying this learning structure to university coursework or the wider program when Hayat requests it; do not expand this project into a university plan now.